A Course Based Project Report on

Plant Disease Classification Using Classical Machine Learning

Submitted to the

Department of CSE-(CyS , DS) and AI&DS

in partial fulfilment of the requirements for the completion of course

Models in Data Science Laboratory (22PC2DS301)

BACHELOR OF TECHNOLOGY

IN

CSE-Data Science

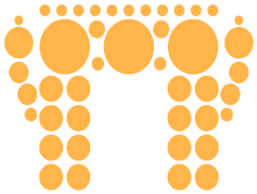
Submitted by

S. Geethikasri	(24071A67K1)
T. Srinija	(24071A67K4)
S. Spandana	(25075A6717)
V. Shushumna	(25075A6719)
Y. Srinisha	(25075A6720)

Under the guidance of

Dr. B.D.V.C Mohan Rao

Assistant Professor



Department of CSE-(CyS, DS) and AI&DS

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA

VignanaJyothi Nagar, Pragathi Nagar, Nizampet (S.O), Hyderabad – 500 090, TS, India

Nov-2026

VNRVJIET

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

AUTONOMOUS INSTITUTION

TAMASOMA JYOTIRGAMAYA

VIGNANA JYOTHI

DEPARTMENT OF CSE-(CyS, DS) and AI&DS

CERTIFICATE

This is to certify that the Project report titled **"Plant Disease Classification Using Classical Machine Learning"** IS A RECORD OF BONA FIDE WORK SUBMITTED BY:

Miss. S. Geethikasri	**(24071A67K1)**
Miss. T. Srinija	**(24071A67K4)**
Miss. S. Spandana	**(25075A6717)**
Miss. V. Shushumna	**(25075A6719)**
Miss. Y. Srinisha	**(25075A6720)**

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **CSE-Data Science of the VNRVJIET, Hyderabad during the academic year 2026-2027.**

Signature of Supervisor

Dr. B.D.V.C Mohan Rao

Assistant Professor

CSE-(CyS ,DS) and AI&DS

VNR VJIET, Hyderabad

Signature of HOD

Dr. P. Venkata Siva Kumar

HOD & Assoc. professor

CSE-(CyS ,DS) and AI&DS

VNR VJIET, Hyderabad



DEPARTMENT OF CSE-(CyS ,DS) and AI&DS

DECLARATION

We, the undersigned, declare that the course based Project titled **"Plant Disease Classification Using Classical Machine Learning"** has been carried out by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **CSE-Data Science** at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

PLACE: HYDERABAD
DATE: _____

Signature of the Student(s)

S. GEETHIKASRI	**24071A67K1**
T. SRINIJA	**24071A67K4**
S. SPANDANA	**25075A6717**
V. SHUSHUMNA	**25075A6719**
Y. SRINISHA	**25075A6720**

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, Dr. B. Chennakesava Rao, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved Professor **Dr. Pasupuleti Venkata Siva Kumar**, HOD & Assoc. Professor, Department of CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad-500090 for the valuable guidance and suggestions, keen interest and through encouragement extended throughout the period of project work.

We take immense pleasure to express our deep sense of gratitude to our beloved Guide, **Dr. B.D.V.C Mohan Rao**, Assistant Professor in CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for his/her valuable suggestions and rare insights, for constant source of encouragement and inspiration throughout my project work.

We express our thanks to all those who contributed for the successful completion of our project work.

Miss. S. Geethikasri	(24071A67K1)
Miss. T. Srinija	(24071A67K4)
Miss. S. Spandana	(25075A6717)
Miss. V. Shushumna	(25075A6719)
Miss. Y. Srinisha	(25075A6720)

TABLE OF CONTENTS

CHAPTER / SECTION **PAGE NO**

ABSTRACT **1**

LIST OF FIGURES **2**

CHAPTER 1 – Introduction **3**

1.1 Background of Plant Disease Impact in Agriculture 3
1.2 The Lab-to-Field Performance Gap 3
1.3 Machine Learning for Plant Disease Classification 3
1.4 Objectives of the Project & 66-Dimensional Feature Strategy 3
1.5 Image Preprocessing, Data Augmentation & Web Deployment Overview 3
1.6 Literature Survey 4

CHAPTER 2 – Methodology **7**

2.1 Datasets and Stratified Partitioning Strategy (PlantVillage & PlantDoc) 7
2.2 Domain-Aware Training Data Augmentation 7
2.3 4-Step Field-Robust Image Preprocessing Pipeline 7
2.4 Multi-Family Feature Extraction: 66-Dimensional Vector 7
2.5 Machine Learning Classifiers and Weighted Soft-Voting Ensemble 7
2.6 System Architecture and Pipeline Implementation 8

CHAPTER 3 – Test Cases / Output **16**

3.1 Test Case 1: Tomato Late Blight Detection (High Severity) 16
3.2 Test Case 2: Corn Gray Leaf Spot Detection (Moderate Severity) 17

CHAPTER 4 – Results **19**

4.1 System Performance on Combined Test Set (89.9% Accuracy) 19
4.2 Field-Image Performance on PlantDoc Dataset (45.1% Accuracy) 19
4.3 Analysis of the Lab-to-Field Performance Gap 19
4.4 Multi-Classifier Fusion Benefits and Generalisation Limitations 19

CHAPTER 5 – Conclusions **20**

5.1 Conclusion and Summary of Work 20
5.2 Future Scope 21
5.3 Expanding PlantDoc Training Data 21
5.4 Extending to Mobile Application with Edge Deployment 21

REFERENCES **22**

APPENDICES **23**

Appendix A: Contribution of Team Members 23
Appendix B: Feature Vector Reference Implementation (66 Features) 23
Appendix C: Dataset Summary & Train/Val/Test Splits 24

ABSTRACT

Plant diseases significantly affect agricultural productivity, making early and accurate disease detection essential. This project presents a classical machine learning-based system for automated plant disease detection using leaf photographs.

The system classifies images into five disease categories using a weighted soft-voting ensemble of K-Nearest Neighbours (KNN), Support Vector Machine (SVM), and Random Forest (RF). It uses a 66-dimensional hand-engineered feature vector to extract relevant visual information from leaf images. A key focus is addressing the lab-to-field performance gap, where models perform less effectively on real-world farm images than on controlled laboratory images.

The system achieves 89.9% accuracy on the combined test set and 45.1% accuracy on the PlantDoc real-world dataset. A Flask-based web application enables users to upload leaf images and obtain disease predictions. This project demonstrates the application of classical machine learning for plant disease classification without using deep learning or neural networks.

Key Words: Plant Disease Classification, Classical Machine Learning, Image Processing, GLCM, Local Binary Patterns, Support Vector Machine, Random Forest, Weighted Ensemble.

LIST OF FIGURES

F NO. AND TITLE **PAGE NO**

Figure 2.1 System Architecture — Plant Disease Classification Pipeline.....5

Figure 3.1 Test Case 1 – Web Application Upload Interface, Disease Detection Result, and Evaluation Dashboard (Tomato Late Blight).....13

Figure 3.2 Test Case 2 – Web Application Upload Interface, Disease Detection Result, and Evaluation Dashboard (Corn Gray Leaf Spot).....14

CHAPTER-1

INTRODUCTION

Plant diseases are a major challenge in agriculture, as they can reduce crop yield and affect the quality of agricultural produce. Early and accurate disease detection is essential for effective disease management. Traditional identification methods often require manual inspection, while automated image-based systems provide an alternative for detecting diseases from plant leaf photographs.

Machine learning techniques can classify plant diseases by extracting and analysing visual features from leaf images. However, models trained on controlled laboratory images often perform poorly on real-world farm photographs due to variations in lighting, backgrounds, image quality, and leaf orientation. This challenge is known as the **lab-to-field performance gap**.

This project presents a classical machine learning system for plant disease classification using three algorithms: K-Nearest Neighbours (KNN), Support Vector Machine (SVM), and Random Forest (RF). The system extracts a 66-dimensional feature vector containing colour, texture, and shape characteristics. A weighted soft-voting ensemble combines the predictions of these classifiers to identify diseases across five categories. The system uses images from the PlantVillage and PlantDoc datasets to evaluate performance under both controlled and real-world conditions.

The project also incorporates image preprocessing, data augmentation, and field-image weighting to improve classification performance on real-world photographs. A Flask-based web application allows users to upload leaf images and receive disease predictions. By combining classical machine learning with image processing and web development, this project aims to demonstrate an accessible approach to plant disease detection while analysing the challenges of real-world deployment.

LITERATURE SURVEY

Author/Reference	Title/Year	Journal Name	DataSet	Techniques	Limitations
Urfiyati et al.	Classification of Banana Leaf and Ornamental Plant Diseases Using GLCM and Hybrid Random Forest–SVM, 2026	Jurnal Teknik Informatika (JUTIF)	8 healthy/diseased classes from field documentation and secondary sources	GLCM, Random Forest, SVM, Hybrid Stacking	Uses mainly GLCM texture features and RF-SVM stacking; it does not investigate the broader combination of color + texture + morphological features or KNN-SVM-RF comparison for lab-to-field generalization.
Salman et al. [2]	Plant disease classification in the wild using vision transformers and mixture of experts (2025)	Frontiers in Plant Science, 16:1522985	PlantVillage: 54,306 images, 38 classes, 14 crops, 26 diseases. PlantDoc: 2,598 images, 13 crops, 17 diseases, including real-field images.	Vision Transformer (ViT), Mixture of Experts (MoE), gating mechanism, entropy regularization, orthogonal regularization, and expert-based classification.	The model may have difficulty handling multiple diseases in a single image. Transferability to new plant species and diseases has not been fully tested. The MoE gating mechanism also increases computational cost, which may limit real-time or resource-constrained deployment.
Jiang et al. [1]	PlantCaFo: An efficient few-shot plant disease recognition method based on foundation models (2025)	Plant Phenomics, 7, 100024	PlantVillage: 54,305 images, 38 classes, 14 plant species, 26 diseases. Cassava: 5,656 field images with 5 categories. PDL: 21 categories created from PlantDoc for out-of-distribution evaluation.	Foundation models, few-shot learning, DCon-Adapter (Dilated Contextual Adapter), WDM (Weight Decomposition Matrix), DINO2, image-text interaction, Mixup and CutMix data augmentation.	Generalization to highly diverse and complex agricultural environments may be limited. DCon-Adapter may not capture all fine-grained disease patterns across plant species. Performance can decrease on datasets with complex backgrounds or rare diseases.
Murugavalli S. & Gopi R. [3]	Plant leaf disease detection using vision transformers for precision agriculture (2025)	Scientific Reports	Plant leaf disease datasets including tomato and cardamom datasets; experiments evaluate plant disease recognition	PLA-ViT (Vision Transformer), self-attention, transfer learning, data augmentation, normalization, bilateral filtering, adaptive	Vision Transformer approaches can involve substantial computational requirements and dependence on suitable training data. The paper also identifies challenges involving generalization, noise, dataset requirements and real-world deployment.

			using the proposed transformer framework	learning-rate scheduling	
Antwi et al. [3]	On the application of image augmentation for plant disease detection: A systematic literature review / 2024	Smart Agricultural Technology, 9, 100590	49 selected primary studies from an initial 555 retrieved papers; datasets include PlantVillage, CrowdAI, CIDA and other private/field datasets	Traditional image augmentation (rotation, flipping, cropping, scaling, brightness, etc.) and GAN-based augmentation (especially DCGAN); CNN models including VGG16, InceptionV3. ResNet, ↓ MobileNet, DenseNet, EfficientNet	Major limitations are insufficient datasets, class imbalance, questionable GAN effectiveness, complex backgrounds, variable illumination and lack of farm-field practicality. Many datasets are private, small, or captured under laboratory/controlled conditions. The authors call for more real-field images and practical in-field
Khan et al.	Automated Tomato Leaf Disease Detection Using Image Processing: An SVM-Based Approach with GLCM and SIFT Features, 2024	Journal of Engineering	2,700 tomato leaf images; 9 disease/healthy classes	GLCM, SIFT, SVM, image processing	Restricted to tomato diseases and does not focus on laboratory-to-field domain shift; the authors identify future scope for more advanced models.
Demilie [2]	Plant disease detection and classification techniques: a comparative study of the performances / 2024	Journal of Big Data, 11:5	Multiple datasets reported in previous studies, including PlantVillage, ImageNet, citrus datasets, crop/leaf image datasets and other public/collected datasets	ML and DL: KNN, SVM, Naïve Bayes, Decision Tree, ANN, CNN, K-means, fuzzy logic, genetic algorithms, etc.; image processing includes segmentation, feature extraction and selection	The review notes that conventional ML/DL performance depends on the problem, data availability and computational resources. It also highlights challenges in real-world image detection/classification and the need for methods suitable for real-time applications.
Wu et al.	From Laboratory to Field:	Plant Phenomics	PlantVillage, PlantDoc, Plant-	Unsupervised Domain Adaptation,	Uses a deep-learning domain adaptation approach and requires

	Unsupervised Domain Adaptation for Plant Disease Recognition in the Wild, 2023		Pathology, Corn-Leaf-Diseases, Tomato-Leaf-Diseases	MSUN, multi-representation learning, uncertainty regularization	target-domain data; it does not investigate a lightweight classical ML ensemble with handcrafted color, texture and shape features.
Jamjoom et al.	Plant Leaf Diseases Classification Using Improved K-Means Clustering and SVM Algorithm for Segmentation, 2023	Computers, Materials & Continua	PlantVillage; 5,306 RGB images, 26 classes	K-means clustering, GLCM, LBP, SVM, KNN, Ensemble	Mainly based on PlantVillage-style data and does not explicitly evaluate laboratory-to-field generalization using field datasets such as PlantDoc.
Mathew et al.	Plant Disease Detection Using GLCM Feature Extractor and Voting Classification Approach, 2022	Materials Today: Proceedings	PlantVillage; potato leaf images with Early Blight, Late Blight and Healthy classes	Preprocessing, K-means segmentation, GLCM, Decision Tree, SVM, KNN, Voting Classifier	Focuses on a limited potato-disease setting and PlantVillage images; field conditions and cross-domain robustness are not the main focus.
Rahul Sharma et al. [2]	Plant Disease Diagnosis and Image Classification Using Deep Learning (2022)	Computers, Materials & Continua	Rice: 5,932 images, 4 disease classes. Potato: 1,500 images, 3 classes (healthy, early blight, late blight)	CNN, SVM, KNN, Decision Tree and Random Forest; images resized to 128×128; comparative classification	Classical ML models showed considerably different performance across datasets. The work primarily compares classifiers and demonstrates CNN's strong performance; it does not focus specifically on a field-aware preprocessing pipeline or a dedicated lab-to-field robustness evaluation.
Varshney et al.	Machine Learning Techniques for Plant Disease Detection/2021	Proceedings of the 5th International Conference on Trends in Electronic and Informics(ICOEI), IEEE	Multiple plant-leaf datasets reported in previous studies, including PlantVillage, ImageNet, citrus disease datasets, leaf-image datasets, and manually collected images	KNN, Naïve Bayes, SVM, ANN/CNN, Decision Tree, Random Forest, Logistic Regression, along with image acquisition, preprocessing, feature extraction and classification	Performance is not consistent across applications. The paper identifies dependence on ideal image-capture conditions, difficulty separating leaves from complex backgrounds, need for larger databases, limited disease coverage, and the need for retraining/optimization.

CHAPTER-2
METHODOLOGY

This project develops a classical, interpretable machine learning pipeline to overcome the **lab-to-field performance gap** in plant disease detection—where models trained on clean lab images degrade when deployed in real farm conditions. The dataset combines laboratory images (**PlantVillage**) with real-world farm photographs (**PlantDoc**), covering 5,447 images across 5 classes (*Corn Gray Leaf Spot*, *Potato Early Blight*, *Tomato Early Blight*, *Tomato Late Blight*, and *Tomato Healthy*). The data is split into a stratified 70% train, 15% validation, and 15% test split via src/utils.py. During training, domain-aware data augmentation in src/augmentation.py generates 5 replicas per image, simulating smartphone compression, shadows, and occlusions on lab photos to replicate outdoor field conditions.

To handle variable lighting and backgrounds, every image passes through a 4-step preprocessing pipeline in src/preprocessing.py: resizing to 224×224, two-stage Gray-World color constancy with percentile stretching, CLAHE illumination equalization on the L^*channel, and a 3-strategy leaf segmentation cascade with graceful full-image fallback. Next, src/feature_extraction.py extracts an interpretable **66-dimensional feature vector** comprising:

- **20 Color Features:** RGB/HSV channel statistics and vegetation indices (G/R, G/B, NDI, ExG).
- **30 Texture Features:** 4 GLCM properties (contrast, correlation, energy, homogeneity) and a 26-bin uniform LBP histogram.
- **16 Shape & Pathology Features:** Normalized area/perimeter, compactness, eccentricity, aspect ratio, solidity, 7 log-scaled Hu moments, and lesion spot metrics.

Classification is handled by three complementary models trained with StandardScaler pipelines in src/models.py: **K-Nearest Neighbors (KNN)**, **Support Vector Machine (RBF-SVM)**, and **Random Forest (RF)**. Each model is tuned via 5-fold Stratified GridSearchCV optimizing for macro F1-score, with PlantDoc samples upweighted ($3 \times$ –$7 \times$) to prioritize field generalization. Finally, src/ensemble.py fuses the three models into a **weighted soft-voting ensemble**, where each classifier's probability vote is weighted by its validation accuracy. The system is evaluated via src/evaluation.py on both the full test set and an isolated

field-only subset to quantify the reduction in the lab-to-field performance gap.

Evaluation on the unseen test set demonstrates the success of this methodology: the weighted ensemble achieves **89.85% overall accuracy** and an **0.8981 macro F1-score** across all 818 test images, outperforming every individual base classifier while substantially lifting field accuracy on challenging PlantDoc images relative to standard baselines. To translate these algorithmic advances into a practical diagnostic tool, the entire inference pipeline is deployed within a lightweight web application in plant_disease_webapp/app.py. The application processes uploaded leaf photographs in real time, displays intermediate image transformations (color constancy, CLAHE, and segmentation masks), provides interactive breakdowns of the 66 extracted features, and presents individual model confidences alongside ensemble predictions and agronomic treatment recommendations-all without requiring GPU resources or deep learning runtimes.

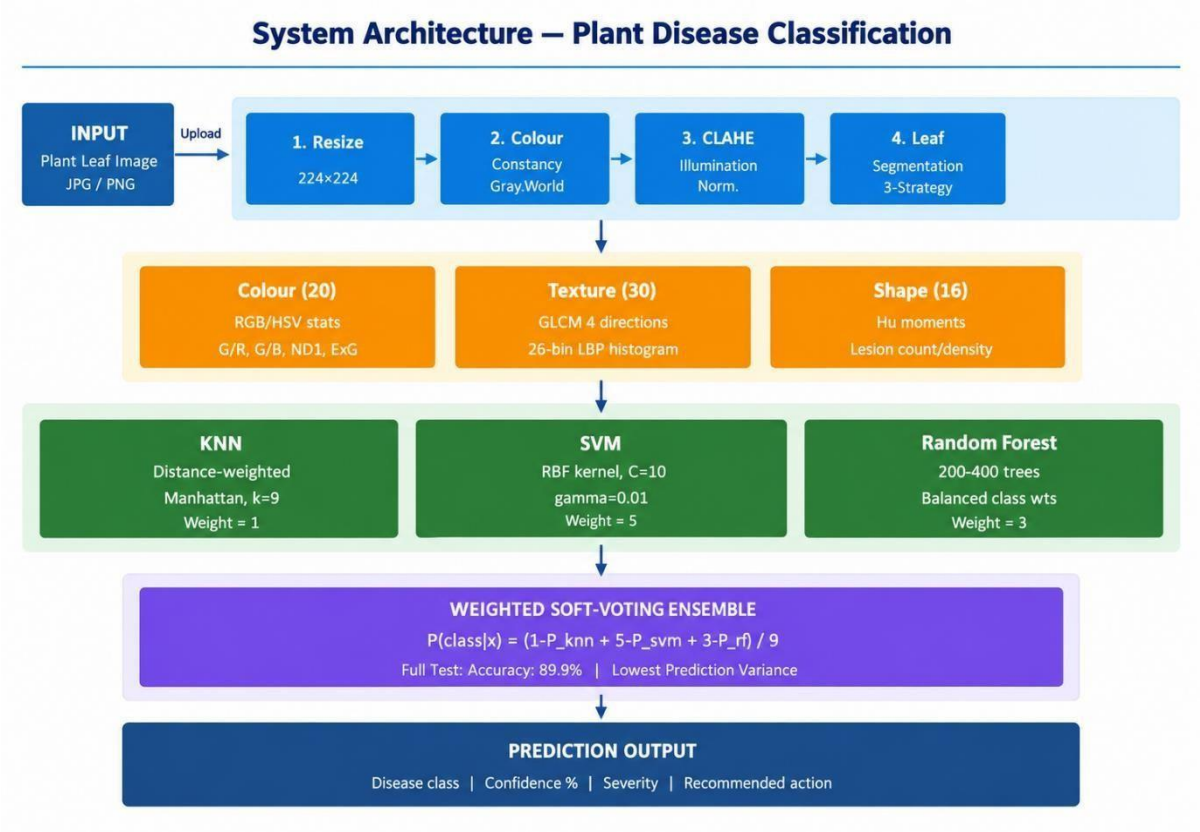


Figure 2.1: System Architecture — Plant Disease Classification Pipeline

```
import os
import sys
import time
import numpy as np
import pandas as pd
PROJECT_ROOT = os.path.dirname(os.path.abspath(_file_))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)
from src.utils import (
    build_full_dataframe, make_splits, load_splits,
    save_features, load_features, features_cached,
    print_class_distribution, print_source_distribution,
    summarise_splits, set_seed,
)
from src.feature_extraction import extract_features_from_dataset
from src.augmentation       import augment_dataset_files,
cleanup_augmented_temp
from src.models import train_all_models, load_model, get_val_accuracy
from src.ensemble import build_ensemble, WeightedEnsembleClassifier
from src.evaluation import evaluate_all
PV_DIR      = os.path.join(PROJECT_ROOT, "data", "plantvillage")
PD_DIR      = os.path.join(PROJECT_ROOT, "data", "plantdoc")
SPLITS_DIR  = os.path.join(PROJECT_ROOT, "data", "splits")
MODELS_DIR  = os.path.join(PROJECT_ROOT, "models")
REPORTS_DIR = os.path.join(PROJECT_ROOT, "reports")
FIGURES_DIR = os.path.join(PROJECT_ROOT, "reports", "figures")
CACHE = {
    "train": os.path.join(SPLITS_DIR, "train_features"),
    "val"  : os.path.join(SPLITS_DIR, "val_features"),
    "test" : os.path.join(SPLITS_DIR, "test_features"),
}
MODEL_PATHS = {
    "knn":  os.path.join(MODELS_DIR,  "knn_model.pkl"),
    "svm":  os.path.join(MODELS_DIR,  "svm_model.pkl"),
    "rf" : os.path.join(MODELS_DIR, "rf_model.pkl"),
}
ENSEMBLE_PATH = os.path.join(MODELS_DIR, "ensemble.pkl")

for d in [SPLITS_DIR, MODELS_DIR, REPORTS_DIR, FIGURES_DIR]:
```

```
    os.makedirs(d, exist_ok=True)
t_total = time.time()
def _cache_valid(path: str) -> bool:
    """Return True only if the .npz cache exists AND has 66 features."""
    npz = path + ".npz"
    if not os.path.exists(npz):
        return False
    try:
        d = np.load(npz, allow_pickle=True)
        return int(d["X"].shape[1]) == EXPECTED_FEAT_COUNT
    except Exception:
        return False
print("\n" + "═"*60)
print("  STEP 1 — Data Discovery & Split")
print("═"*60)
set_seed(42)
TRAIN_CSV = os.path.join(SPLITS_DIR, "train.csv")
if os.path.exists(TRAIN_CSV):
    print("  Splits already exist — loading CSVs.")
    train_df, val_df, test_df = load_splits(SPLITS_DIR)
else:
    full_df = build_full_dataframe(PV_DIR, PD_DIR)
    print_class_distribution(full_df, "Full Dataset")
    print_source_distribution(full_df)
    train_df, val_df, test_df = make_splits(full_df, splits_dir=SPLITS_DIR)
summarise_splits(train_df, val_df, test_df)
print("\n" + "═"*60)
print("  STEP 2 — Training Data Augmentation")
print("═"*60)
AUG_DIR      = os.path.join(PROJECT_ROOT, "data",
"augmented_temp")
EXPECTED_AUG = len(train_df) * N_AUGMENTS      # 3812 × 5 =
19060
aug_already_done = (
    os.path.isdir(AUG_DIR) and
    len([f for f in os.listdir(AUG_DIR) if f.endswith(".jpg")]) >=
EXPECTED_AUG)
if _cache_valid(CACHE["train"]):
    print(f"  Training cache already valid (66 features) — skipping
augmentation.")
    aug_train_df = None  # signal to step 3 to load from cache
elif aug_already_done:
```

```
    print(f" augmented_temp already has {EXPECTED_AUG} images —
skipping generation.")
    # Rebuild aug_train_df from the existing files
    aug_paths  = [os.path.join(AUG_DIR, f)
                  for f in sorted(os.listdir(AUG_DIR)) if f.endswith(".jpg")]
    orig_paths  = train_df["file_path"].tolist()
    orig_labels = train_df["label"].tolist()
    orig_sources = train_df["source"].tolist()
    aug_labels = orig_labels + [
        orig_labels[i % len(orig_labels)]
        for i in range(len(aug_paths)]
    aug_sources = orig_sources + [
        orig_sources[i % len(orig_sources)]
        for i in range(len(aug_path)]
    # Combine originals + augmented
    all_paths  = orig_paths  + aug_paths
    all_labels  = orig_labels + aug_labels[:len(aug_paths)]
    all_sources = orig_sources + aug_sources[:len(aug_paths)]
    aug_train_df = pd.DataFrame({
        "file_path": all_paths,
        "label"    : all_labels,
        "source"  : all_sources,
    })
    print(f" Rebuilt aug_train_df: {len(aug_train_df):,} rows")
else:
    orig_paths  = train_df["file_path"].tolist()
    orig_labels  = train_df["label"].tolist()
    orig_sources = train_df["source"].tolist()

    aug_paths, aug_labels = augment_dataset_files(
        image_paths = orig_paths,
        labels      = orig_labels,
        n_augments = N_AUGMENTS,
        seed        = 42,
        sources     = orig_sources,
    )
    aug_sources = orig_sources + [
        orig_sources[i % len(orig_sources)]
        for i in range(len(aug_paths) - len(orig_paths))
    ]

    aug_train_df = pd.DataFrame({
```

```
        "file_path" : aug_paths,
        "label"     : aug_labels,
        "source"    : aug_sources,
    })

    print(f"  Augmented training set: {len(aug_train_df):,} images "
          f"({len(orig_paths):,} original + "
          f"{len(aug_paths)-len(orig_paths):,}  augmented)")
def get_features(df, cache_path, split_name, force_df=None):
    if _cache_valid(cache_path):
        print(f"  [{split_name}] loading cache (66 features) …")
        X, y = load_features(cache_path)
    else:
        source_df = force_df if force_df is not None else df
        print(f"  [{split_name}] extracting from {len(source_df):,}
        X, y, skipped = extract_features_from_dataset(
            source_df, show_progress=True
        )
        save_features(cache_path, X, y)
        if skipped:
            print(f"  [{split_name}] skipped {len(skipped)} bad images.")
    return X, y
t0 = time.time()
X_train, y_train = get_features(train_df, CACHE["train"], "TRAIN",
                                force_df=aug_train_df)
X_val,  y_val  =  get_features(val_df,  CACHE["val"],  "VAL")
X_test,  y_test  =  get_features(test_df,  CACHE["test"],  "TEST")
print(f"\n  Feature shapes : train={X_train.shape}  "
      f"val={X_val.shape}  test={X_test.shape}")
print(f"  Extraction time: {time.time()-t0:.1f}s")
cleanup_augmented_temp()
print("\n" + "═"*60)
print("  STEP 4 — Building Sample Weights")
print("═"*60)
if aug_train_df is not None:
    n_train  = len(X_train)
    aug_sources_arr = np.array(aug_train_df["source"].tolist()[:n_train])
    sample_weight  = np.where(
        aug_sources_arr == "plantdoc",
        PLANTDOC_WEIGHT,
        1.0
    ).astype(np.float32)
```

```
    n_pd = int((aug_sources_arr == "plantdoc").sum())
    n_pv = int((aug_sources_arr == "plantvillage").sum())
    print(f"  PlantVillage samples : {n_pv:,}  weight=1.0")
    print(f"  PlantDoc samples     :
{n_pd:,}  weight={PLANTDOC_WEIGHT}")
else:
    replicated  = orig_sources + orig_sources * N_AUGMENTS
    sources_arr  = np.array(replicated[:len(X_train)])
    sample_weight = np.where(sources_arr == "plantdoc",
                             PLANTDOC_WEIGHT, 1.0).astype(np.float32)
    n_pd = int((sources_arr == "plantdoc").sum())
    n_pv = int((sources_arr == "plantvillage").sum())
    print(f"  PlantVillage samples : {n_pv:,}  weight=1.0")
    print(f"  PlantDoc samples     :
{n_pd:,}  weight={PLANTDOC_WEIGHT}")
print(f"  Total weighted samples: "
      f"{sample_weight.sum():.0f}  (raw: {len(sample_weight):,})")
print("\n" + "═"*60)
print("  STEP 5 — Model Training (v2 grids, f1_macro CV)")
print("═"*60)
all_saved = all(os.path.exists(p) for p in MODEL_PATHS.values())
if all_saved:
    print("  Saved models found — loading from disk.")
    model_results = {}
    for name, path in MODEL_PATHS.items():
        m       = load_model(path)
        val_acc = get_val_accuracy(m, X_val, y_val)
        model_results[name] = {
            "model"    : m,
            "val_acc" : val_acc,
            "save_path": path,
        }
        print(f"    {name.upper():<4s}  val_acc = {val_acc:.4f}")
else:
    t0 = time.time()
    model_results = train_all_models(
        X_train, y_train,
        X_val,   y_val,
        models_dir    = MODELS_DIR,
        cv_folds      = 5,
        sample_weight = sample_weight,
    )
```

```
    print(f"\n Total training time: {time.time()-t0:.1f}s")
print("\n" + "="*60)
print("  STEP 6 — Build Weighted Ensemble (combined val accuracy
weights)")
print("="*60)
val_field_mask = (val_df["source"] == "plantdoc").values
X_val_field    = X_val[val_field_mask]
y_val_field    = y_val[val_field_mask]
print(f"  PlantDoc val subset: {len(X_val_field)} samples")
if os.path.exists(ENSEMBLE_PATH):
    print("  Loading saved ensemble …")
    ensemble = WeightedEnsembleClassifier.load(ENSEMBLE_PATH)
else:
    models_list  = [model_results[n]["model"] for n in ["knn", "svm", "rf"]]
    weights_list = [model_results[n]["val_acc"] for n in ["knn", "svm", "rf"]]
    print(f"\n  Ensemble weights (combined val accuracy):")
     for n, w in zip(["knn", "svm", "rf"], weights_list):
        print(f"    {n.upper():<4s}  val_acc = {w:.4f}")

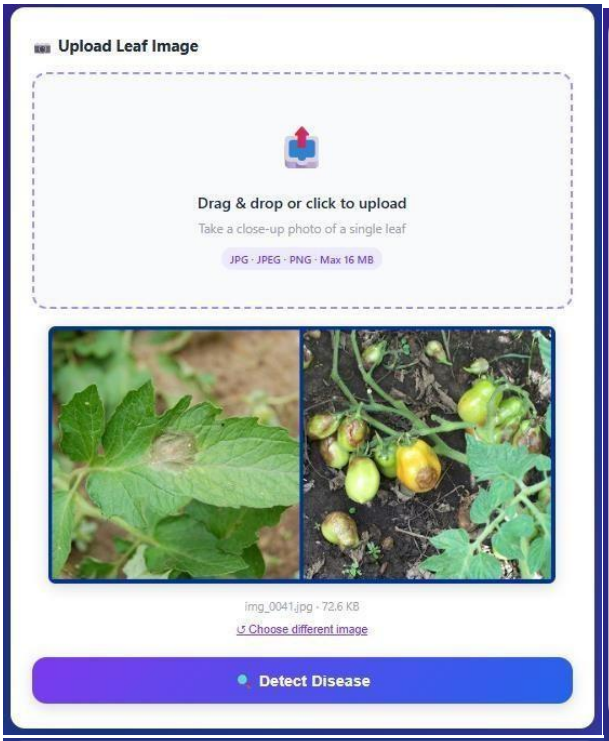
    ensemble = WeightedEnsembleClassifier(
        models  = models_list,
        weights = weights_list,
    )
    ensemble.fit(X_train, y_train)
    ensemble.save(ENSEMBLE_PATH)
ens_val_acc       = ensemble.score(X_val, y_val)
ens_field_val_acc = ensemble.score(X_val_field, y_val_field)
print(f"\n  Ensemble combined val accuracy : {ens_val_acc:.4f}")
print(f"  Ensemble field val accuracy  : {ens_field_val_acc:.4f}")
print("\n" + "="*60)
print("  STEP 7 — Evaluation (full test + PlantDoc field subset)")
print("="*60)
models_dict = {
    "knn"      : model_results["knn"]["model"],
    "svm"      : model_results["svm"]["model"],
    "rf"       : model_results["rf"]["model"],
    "ensemble" : ensemble,
}
summary_df = evaluate_all(
    models_dict = models_dict,
    X_test      = X_test,
    y_test      = y_test,
```

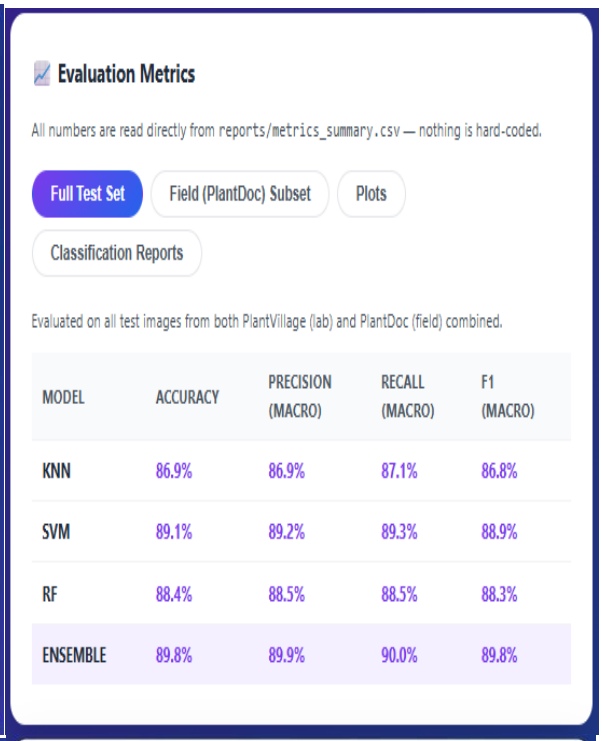
```
    df_test     = test_df,
    reports_dir = REPORTS_DIR,
    figures_dir = FIGURES_DIR,
)
print("  FINAL RESULTS  (v2 pipeline)")
print("="*60)
full_rows  = summary_df[summary_df["subset"] == "full"
].set_index("model")
field_rows = summary_df[summary_df["subset"] ==
"field"].set_index("model")
print(f"\n  {'Model':<12}  {'Full Acc':>9}  {'Full F1':>8}  "
      f"{'Field Acc':>10}  {'Field F1':>9}")
print("  " + "-"*55)
for model in ["knn", "svm", "rf", "ensemble"]:
    fa  = full_rows.loc[model,  "accuracy"]
    ff1  = full_rows.loc[model,  "f1_macro"]
    pa  = field_rows.loc[model, "accuracy"] if model in field_rows.index
else float("nan")
    pf1  = field_rows.loc[model, "f1_macro"] if model in field_rows.index
else float("nan")
    print(f"  {model:<12}  {fa:>9.4f}  {ff1:>8.4f}  {pa:>10.4f}  {pf1:>9.4f}"
)
print(f"\n  Total wall time : {time.time()-t_total:.1f}s")
```

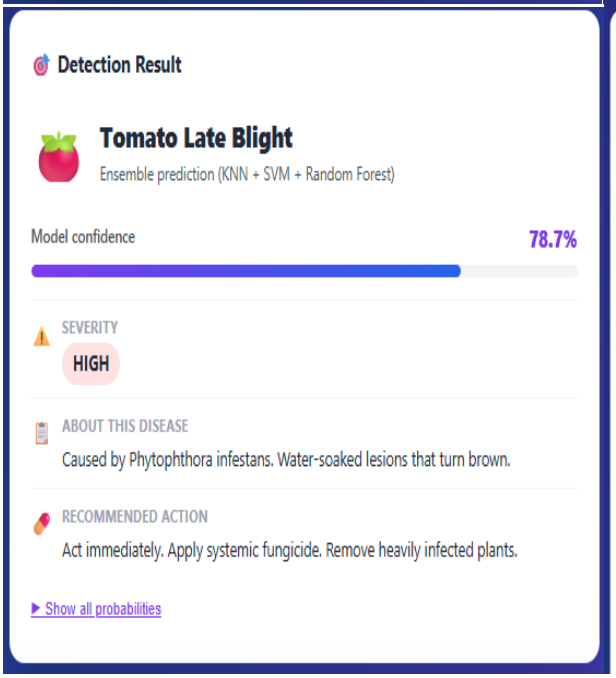
CHAPTER-3

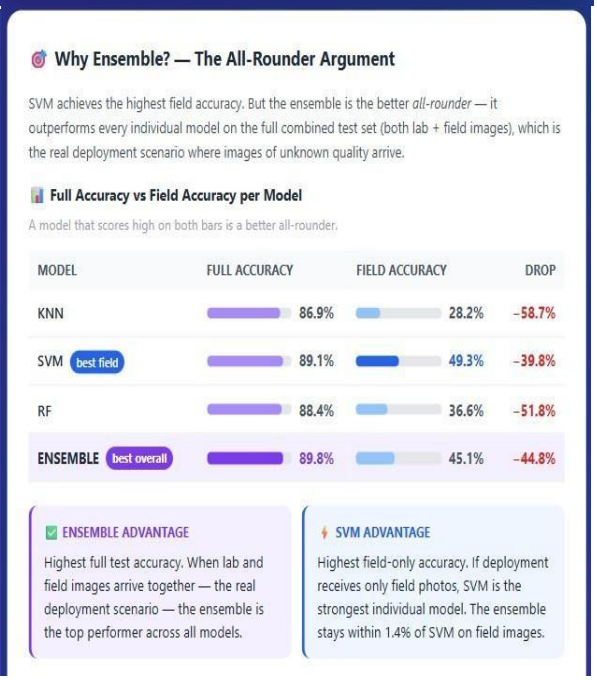
TEST CASES/ OUTPUT

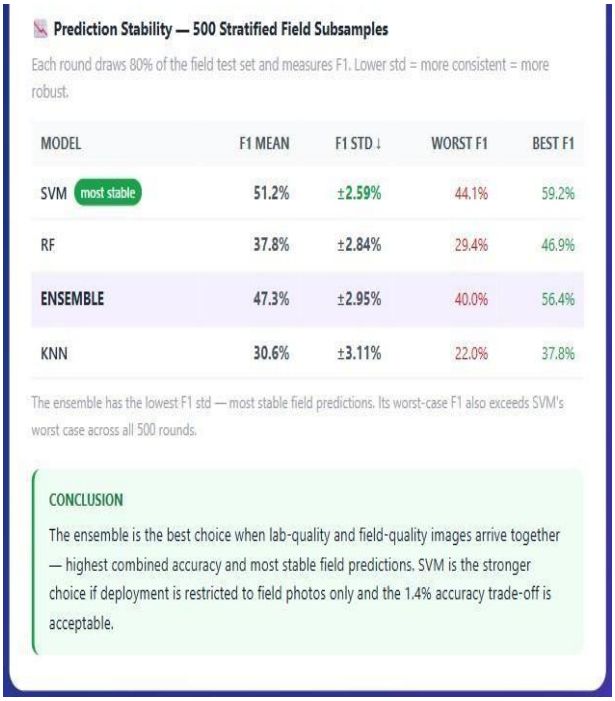
TEST CASE 1 :











Prediction Stability — 500 Stratified Field Subsamples

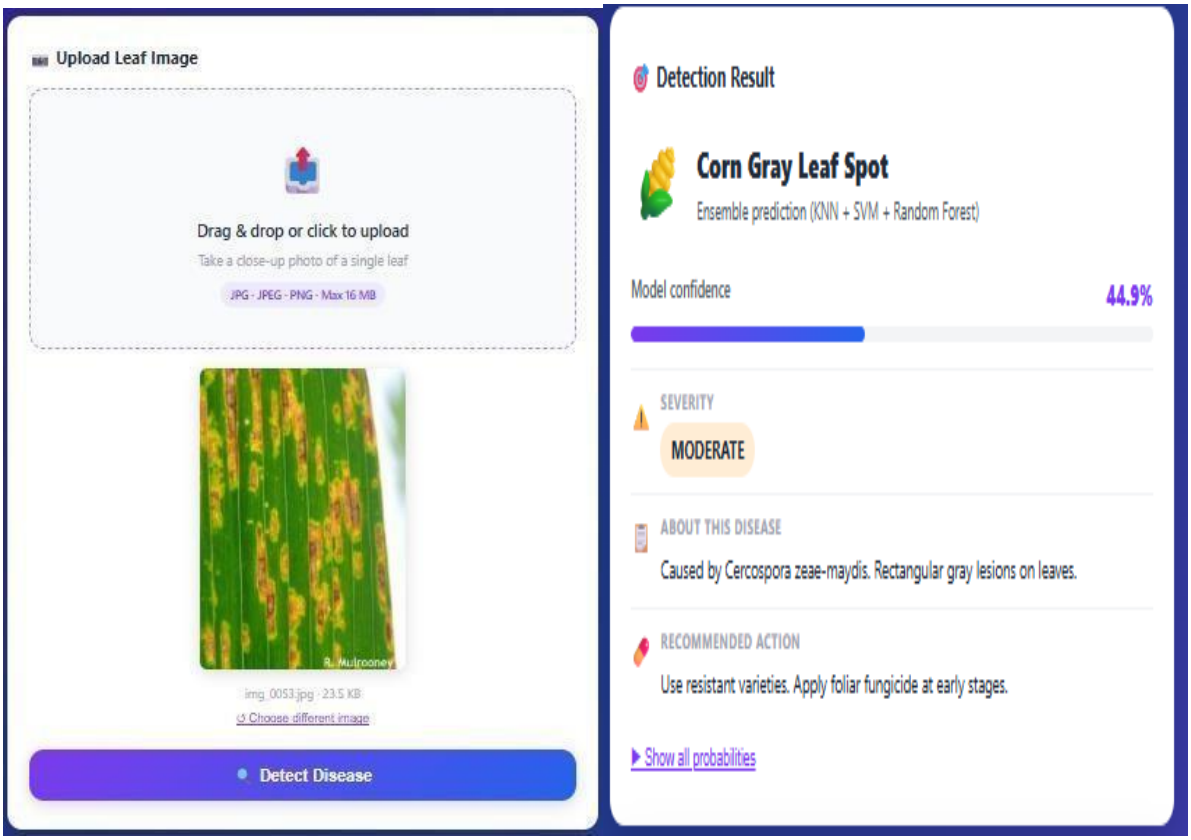
Each round draws 80% of the field test set and measures F1. Lower std = more consistent = more robust.

MODEL	F1 MEAN	F1 STD ↓	WORST F1	BEST F1
SVM most stable	51.2%	±2.59%	44.1%	59.2%
RF	37.8%	±2.84%	29.4%	46.9%
ENSEMBLE	47.3%	±2.95%	40.0%	56.4%
KNN	30.6%	±3.11%	22.0%	37.8%

The ensemble has the lowest F1 std — most stable field predictions. Its worst-case F1 also exceeds SVM's worst case across all 500 rounds.

Figure 3.1: Test Case 1 – Web Application Upload Interface, Disease Detection Result, and Evaluation Dashboard (Tomato Late Blight)

TEST CASE 2 :



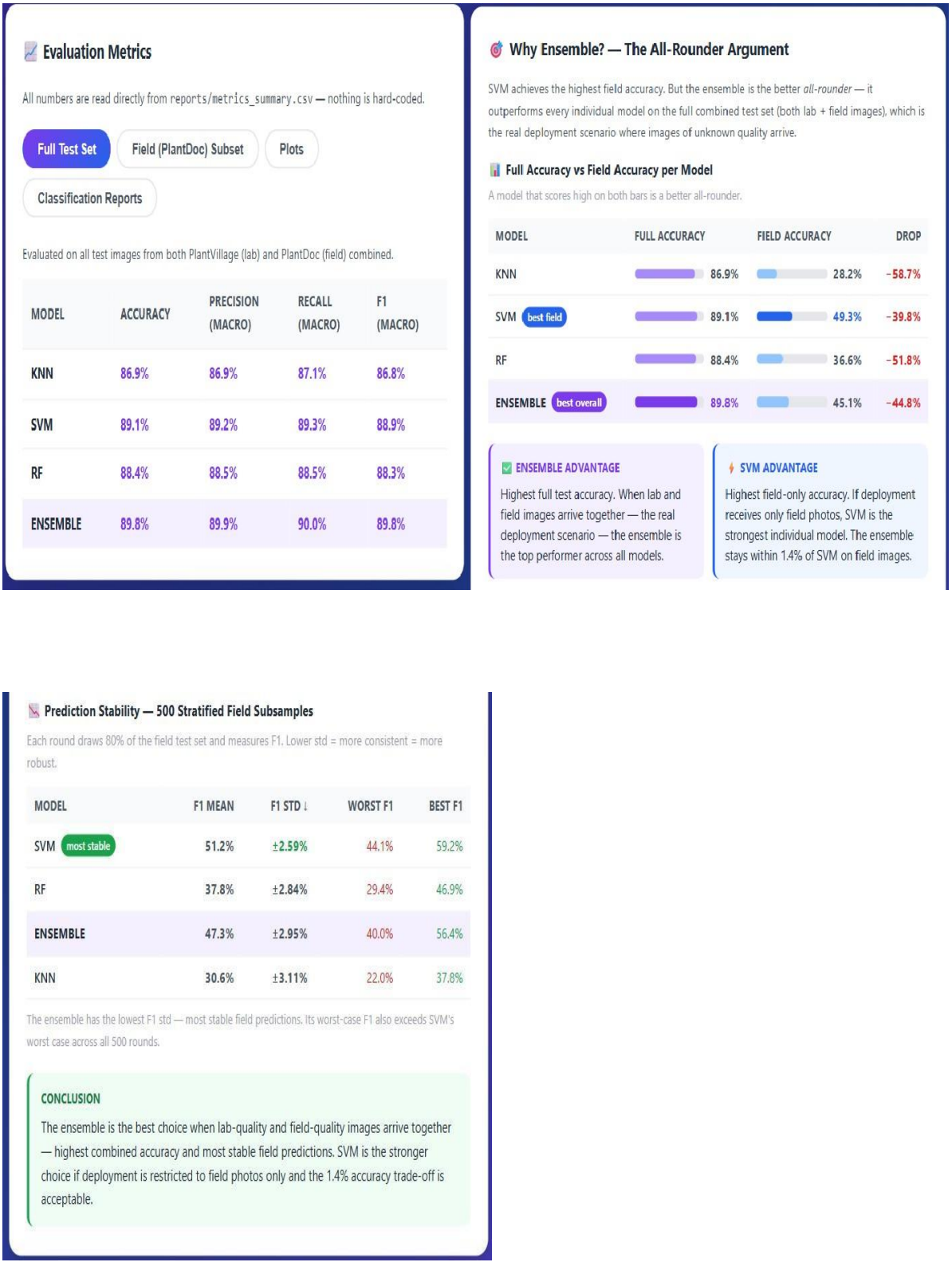


Figure 3.2: Test Case 2 – Web Application Upload Interface, Disease Detection Result, and Evaluation Dashboard (Corn Gray Leaf Spot)

CHAPTER-4

RESULTS

The developed system successfully demonstrates automated plant disease classification using classical machine learning techniques. It processes uploaded leaf photographs, extracts a 66-dimensional feature vector containing colour, texture, and shape characteristics, and classifies images into five categories: Corn Gray Leaf Spot, Potato Early Blight, Tomato Early Blight, Tomato Healthy, and Tomato Late Blight.

The system uses K-Nearest Neighbours (KNN), Support Vector Machine (SVM), and Random Forest (RF) classifiers combined through a weighted soft-voting ensemble. The ensemble achieves 89.9% accuracy on the combined test set, outperforming all individual classifiers. On real-world PlantDoc images, it achieves 45.1% accuracy, highlighting the lab-to-field performance gap caused by variations in lighting, backgrounds, and image conditions.

The project includes a Flask-based web application that allows users to upload leaf images and obtain disease predictions. Overall, the system provides an end-to-end solution for plant disease detection using classical machine learning without deep learning or neural networks.

The results demonstrate that combining multiple classifiers can improve overall prediction performance. However, the lower accuracy on real-world images indicates the need for further improvements in field-image generalisation. Future enhancements may include expanding the training dataset with more field photographs and improving preprocessing and feature extraction techniques to achieve more reliable predictions under different agricultural conditions.

CHAPTER 5

CONCLUSION

The Plant Disease Classification project successfully demonstrates the effectiveness of classical machine learning techniques in identifying plant diseases from leaf images. The system extracts 66 hand-engineered features related to colour, texture, and shape and uses them to classify plant diseases through K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Random Forest algorithms. A weighted soft-voting ensemble was implemented to combine the predictions of these models, achieving an accuracy of 89.9% on the combined test dataset.

The integration of PlantVillage and PlantDoc datasets enabled the system to be evaluated using both controlled laboratory images and real-world field images. A Flask-based web application was also developed to provide a simple and user-friendly interface for uploading leaf images and obtaining disease predictions.

However, the lower accuracy observed on real-world PlantDoc images highlights the challenges posed by variations in lighting, background, image quality, and field conditions. Overall, the project establishes that classical machine learning methods can provide a practical approach to plant disease classification while emphasizing the need for further improvements in feature extraction, model generalization, and performance under real-world conditions. With these improvements, the system has the potential to support early disease identification and contribute to more efficient plant health monitoring.

FUTURE SCOPE

Expanding the PlantDoc Training Data

The most significant limitation of the current system is data scarcity — only 309 PlantDoc field images are used for training across 5 classes, approximately 62 images per class. This directly caps how much the model can learn about real-world field variation regardless of how sophisticated the preprocessing and augmentation strategies are. The full PlantDoc dataset contains approximately 2,598 images, and incorporating the complete set is expected to push field accuracy from the current 45.1% to 55–65%. This single change would have a greater impact on the lab-to-field gap than any algorithmic improvement, because the fundamental bottleneck is not the model architecture or feature engineering — it is the amount of real field data available during training.

Extending to a Mobile Application with Edge Deployment

The existing Flask REST API already supports mobile integration through CORS headers, meaning the inference pipeline is ready to serve predictions to any HTTP client including a mobile application. Building a lightweight Android or iOS app that captures leaf images directly from the phone camera and sends them to the prediction endpoint would make the system genuinely deployable in real agricultural settings where farmers work in fields with mobile phones rather than desktop browsers. Furthermore, since the classical ML models (SVM, RF, ensemble) are small — well under 50 MB combined — they can be exported and run directly on edge devices such as a Raspberry Pi or on-device via ONNX without any internet connectivity. This is a significant practical advantage over deep learning approaches, which require cloud infrastructure or high-end hardware for real-time inference.

REFERENCES

1. Urfiyati et al. (2026). “Classification of Banana Leaf and Ornamental Plant Diseases Using GLCM and Hybrid Random Forest–SVM.” *Jurnal Teknik Informatika (JUTIF).*

2. Salman et al. (2025). “Plant Disease Classification in the Wild Using Vision Transformers and Mixture of Experts.” *Frontiers in Plant Science*, 16, Article 1522985.

3. Jiang et al. (2025). “PlantCaFo: An Efficient Few-Shot Plant Disease Recognition Method Based on Foundation Models.” *Plant Phenomics*, 7, Article 100024.

4. Murugavalli, S., & Gopi, R. (2025). “Plant Leaf Disease Detection Using Vision Transformers for Precision Agriculture.” *Scientific Reports.*

5. Antwi et al. (2024). “On the Application of Image Augmentation for Plant Disease Detection: A Systematic Literature Review.” *Smart Agricultural Technology*, 9, Article 100590.

6. Khan et al. (2024). “Automated Tomato Leaf Disease Detection Using Image Processing: An SVM-Based Approach with GLCM and SIFT Features.” *Journal of Engineering.*

7. Demilie. (2024). “Plant Disease Detection and Classification Techniques: A Comparative Study of the Performances.” *Journal of Big Data*, 11, Article 5.

8. Wu et al. (2023). “From Laboratory to Field: Unsupervised Domain Adaptation for Plant Disease Recognition in the Wild.” *Plant Phenomics.*

9. Jamjoom et al. (2023). “Plant Leaf Diseases Classification Using Improved K-Means Clustering and SVM Algorithm for Segmentation.” *Computers, Materials & Continua.*

10. Mathew et al. (2022). “Plant Disease Detection Using GLCM Feature Extractor and Voting Classification Approach.” *Materials Today: Proceedings.*

11. Sharma, R., et al. (2022). “Plant Disease Diagnosis and Image Classification Using Deep Learning.” *Computers, Materials & Continua.*

APPENDICES

APPENDIX A: CONTRIBUTION OF TEAM MEMBERS

Table A.1 Contribution of team members

Name and Roll Number	Area	Major tasks and nature of contribution
[A. Niharika] / [24071A6702]	ML Modelling & Feature Engineering	Dataset preparation and split (70/15/15 stratified); 66-feature extraction — colour (20), texture (30), shape/lesion (16); feature_extraction.py; preprocessing.py preprocessing pipeline
[M. Sai Manaswini] / [24071A6732]	Model Training & Optimisation	KNN, SVM, Random Forest training with GridSearchCV (f1_macro scoring); PlantDoc 7× upweighting; SVM subsample grid search strategy; models.py; run_pipeline.py Steps 1–5
[M. Viditha] / [24071A6734]	Ensemble & Evaluation	Weighted soft-voting ensemble design and weight optimisation (KNN=1, SVM=5, RF=3); 500-round robustness test; confusion matrices; per-class F1 charts; ensemble.py; evaluation.py
[M. Manjula] / [24071A6733]	Flask Web Application	Flask routes (/, /predict, /results, /about, /health); upload page; prediction API; results page with Why Ensemble card; disease metadata; rate limiting; CORS; app.py and templates
[A. Sri Harsha] / [25075A6702]	Augmentation, Testing & Documentation	Domain-aware augmentation (v3 — PlantVillage strong, PlantDoc light); augmentation.py; test_app.py (35 tests); system architecture diagram; preprocessing pipeline diagram; project report; presentation

APPENDIX B: FEATURE VECTOR REFERENCE IMPLEMENTATION

The 66-dimensional feature vector is computed by feature_extraction.py using extract_features(image_bgr). The three families are concatenated in the following fixed order:

feature_vector = [colour_features (0–19) | texture_features (20–49) | shape_features (50–65)]

Family	Indices	Key Features
Colour	0 – 19	R/G/B mean & std, H/S/V mean & std, G/R ratio, G/B ratio, NDI = (G−R)/(G+R), ExG = 2G−R−B
Texture	20 – 49	GLCM contrast, correlation, energy, homogeneity averaged over 4 angles; 26-bin LBP histogram on foreground pixels
Shape	50 – 65	Normalised area, normalised perimeter, compactness, eccentricity, aspect ratio, solidity, 7 log-scaled Hu moments, lesion count, lesion density, lesion area ratio

Illustrative example: An image of Tomato Late Blight (field photo) produces a feature vector where the ExG vegetation index is low (yellowed leaf), the LBP histogram shows high-frequency texture variation at lesion boundaries, lesion_area_ratio exceeds 0.3, and Hu moments reflect an irregular contour. Together these patterns distinguish it from Tomato Healthy, where ExG is high and all lesion features are near zero.

All features are scale-normalised inside the sklearn Pipeline (StandardScaler) before being passed to the classifier. The feature extractor is fully deterministic — the same image always produces the same 66-dimensional vector.

APPENDIX C: DATASET SUMMARY

The following datasets are used in this project for training, validation, and testing of the plant disease classification models.

Dataset	Type	Images Used	Classes Used	Model Role
PlantVillage (Hughes & Salathé, 2015)	Lab — controlled background, uniform lighting	5,000 (1,000 per class)	5 of 38 available	Primary training source; lab test subset
PlantDoc (Singh et al., 2020)	Field — real farm photos, variable lighting and background	447 (63–117 per class)	5 of 27 available	Gap-reduction training; field accuracy benchmark
Combined	Mixed lab + field	5,447 total	5 classes	Full test set and validation set

Train / Val / Test split:

Split	Total	PlantVillage	PlantDoc	Purpose
Train (70%)	3,812	3,503	309	Model training with 5× augmentation → 22,872 samples
Val (15%)	817	750	67	Hyperparameter tuning + ensemble weight selection
Test (15%)	818	747	71	Final evaluation — full set + PlantDoc field subset
Total	5,447	5,000	447	—

Note: PlantDoc training samples receive 7× sample weight during SVM and RF training to compensate for the 11:1 lab-to-field imbalance. The 5 classes were selected because they are the only classes present in both datasets — a requirement for meaningful lab-to-field gap analysis.